

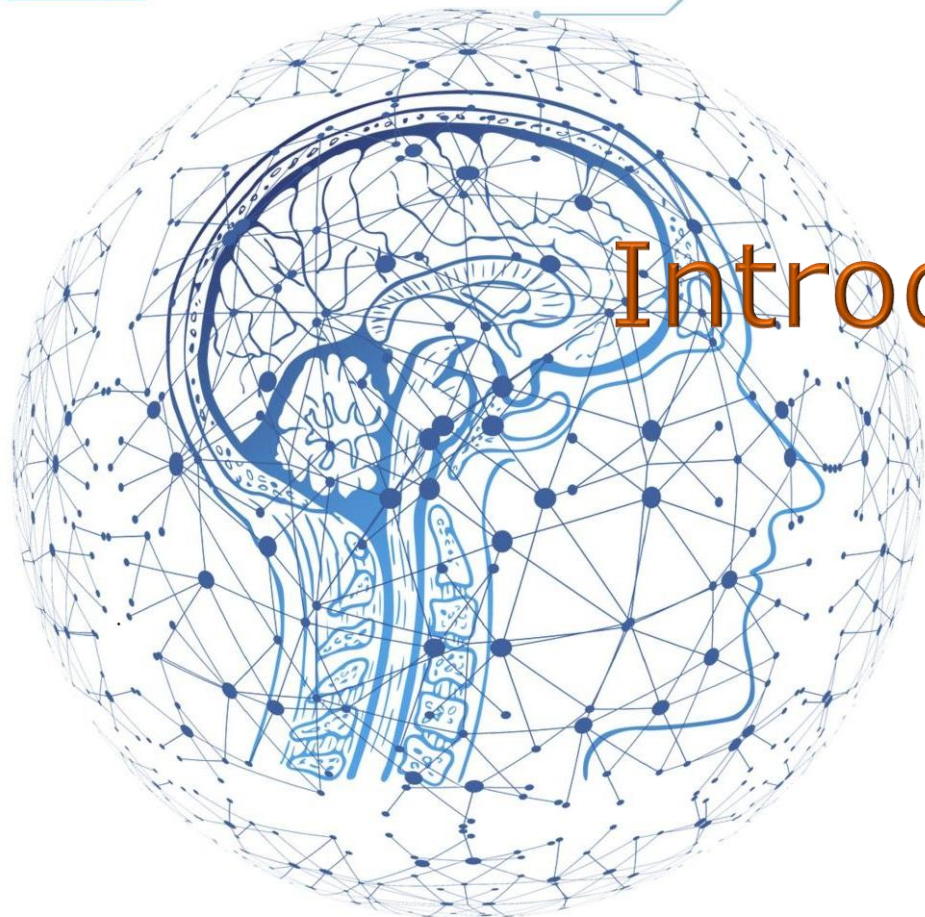
# DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

[www.hoasen.edu.vn](http://www.hoasen.edu.vn)

## Chương 8

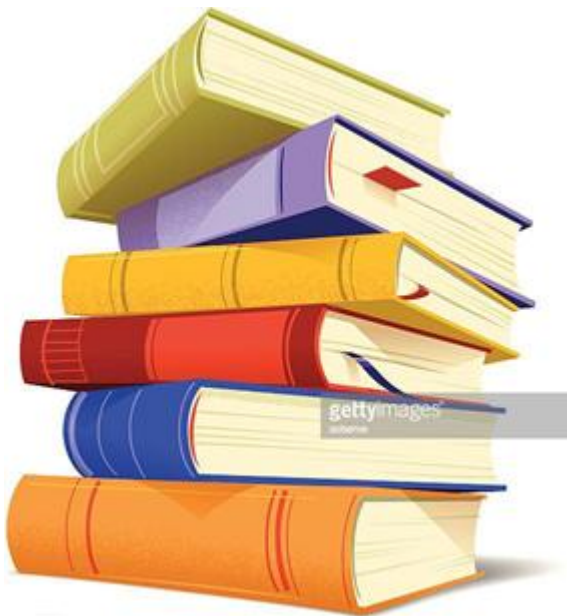
# Introduction to Deep Learning for computer vision Giới thiệu học sâu cho thị giác máy tính



# Introduction to DL for computer vision

## Giới thiệu DL cho thị giác máy tính

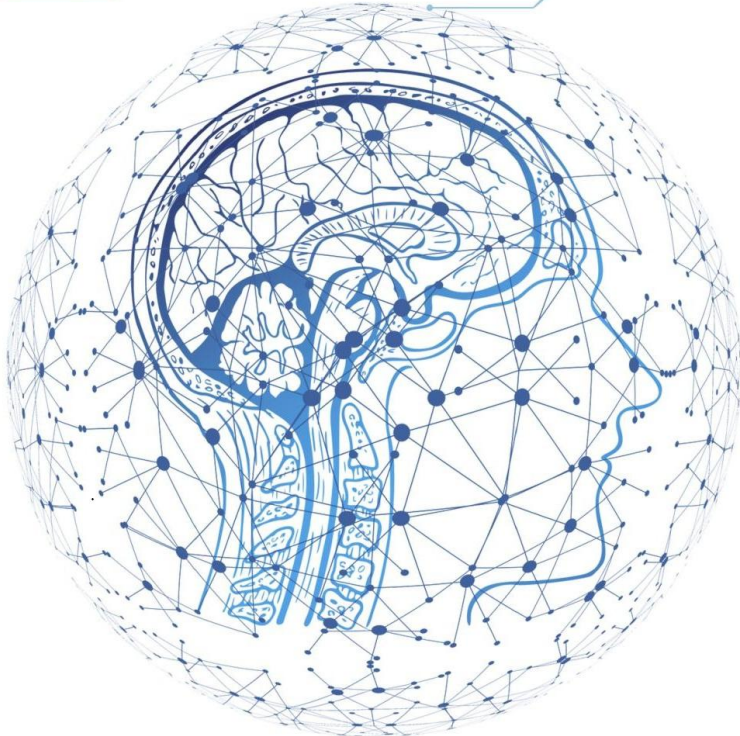
---



- Introduction to convnets - Giới thiệu về Convnet
- Training a convnet from scratch on a small dataset - Huấn luyện convnet từ đầu trên tập dữ liệu nhỏ
- Leveraging a pretrained model - Tận dụng mô hình đã huấn luyện sẵn

# DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



## Introduction to Convnets

### Giới thiệu về Convnet

# Computer vision and Convnet

## Thị giác máy tính và Convnet

---

- **Computer vision** là một trong những thành công sớm và lớn nhất của deep learning.
- Ứng dụng quen thuộc: **Google Photos**, tìm kiếm ảnh, OCR, camera filters, xe tự hành, y tế, robotics.
- Từ 2011-2015, **convolutional neural networks (convnets hay CNNs)** bắt đầu thắng mạnh trong các cuộc thi nhận dạng ảnh.
- Chương này tập trung vào **image classification** với convnet, đặc biệt khi **training data ít**.



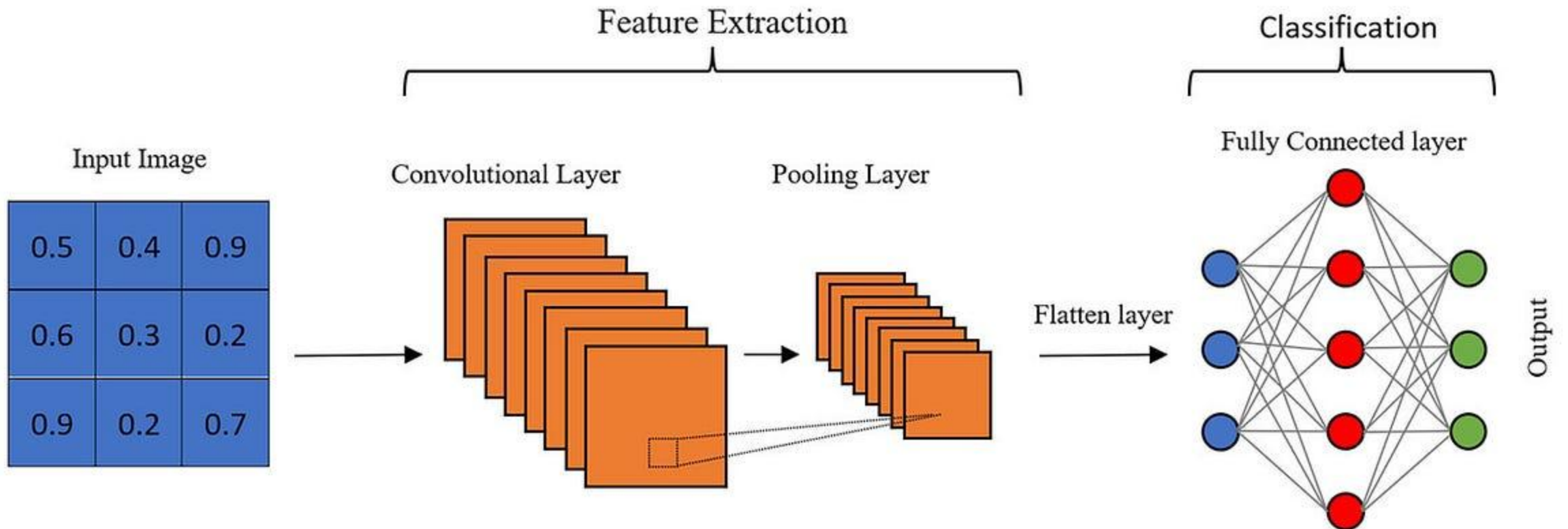
## A first MNIST Convnet

### Convnet đầu tiên với MNIST

---

- Input của convnet có dạng (**height, width, channels**), ví dụ MNIST: **(28, 28, 1)**.
- Kiến trúc mẫu: **Conv2D → MaxPooling2D → Conv2D → MaxPooling2D → Conv2D → Flatten → Dense**.
- Sau các lớp convolution/pooling, tensor 3D được **Flatten** thành vector để đưa vào classifier Dense.
- Kết quả: convnet đơn giản đạt khoảng **99.1% accuracy**, tốt hơn Dense model Chương 2 khoảng **97.8%**.

# Convnet/CNN



# Tích chập

Input                      Kernel                      Output

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

 \* 

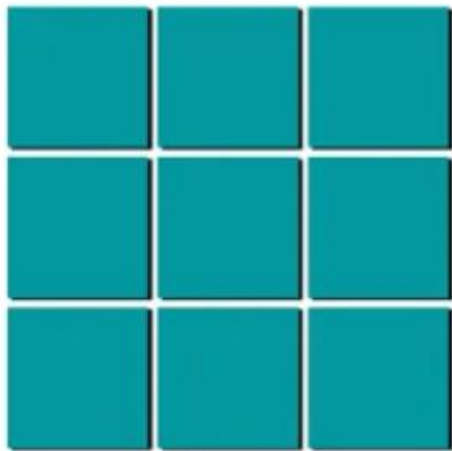
|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

 = 

|    |    |
|----|----|
| 19 | 25 |
| 37 | 43 |

Convolution Operation with a  $2 \times 2$  kernel

# Áp dụng đệm



Input Image

Applying padding  
of 1 on 3x3

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 |   |   |   | 0 |
| 0 |   |   |   | 0 |
| 0 |   |   |   | 0 |
| 0 | 0 | 0 | 0 | 0 |

Padded Image



# Max Pooling

- Chọn giá trị lớn nhất từ mỗi vùng lọc
- Giữ lại các đặc điểm quan trọng như cạnh và kết cấu
- Giảm kích thước không gian của bản đồ đặc trưng
- Thường được sử dụng do hiệu suất cao trong thực tế

|   |   |   |   |
|---|---|---|---|
| 2 | 2 | 7 | 3 |
| 9 | 4 | 6 | 1 |
| 8 | 5 | 2 | 4 |
| 3 | 1 | 2 | 6 |

Max Pool

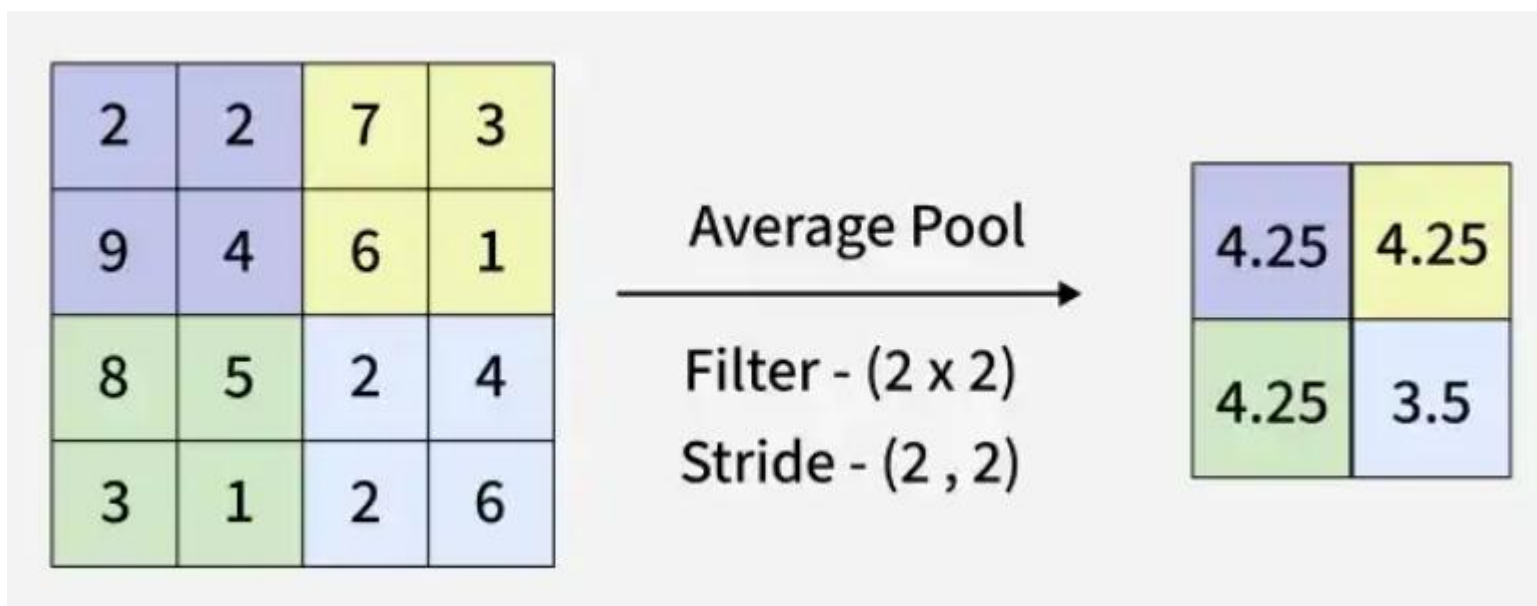
Filter - (2 x 2)

Stride - (2, 2)

|   |   |
|---|---|
| 9 | 7 |
| 8 | 6 |

# Average Pooling

- Tính toán giá trị trung bình từ mỗi vùng lọc
- Thể hiện các đặc điểm tổng thể thay vì các đặc điểm mạnh nhất
- Giảm kích thước không gian của bản đồ đặc trưng
- Tạo ra bản đồ đặc trưng mượt mà hơn so với phương pháp gộp tối đa (max pooling)

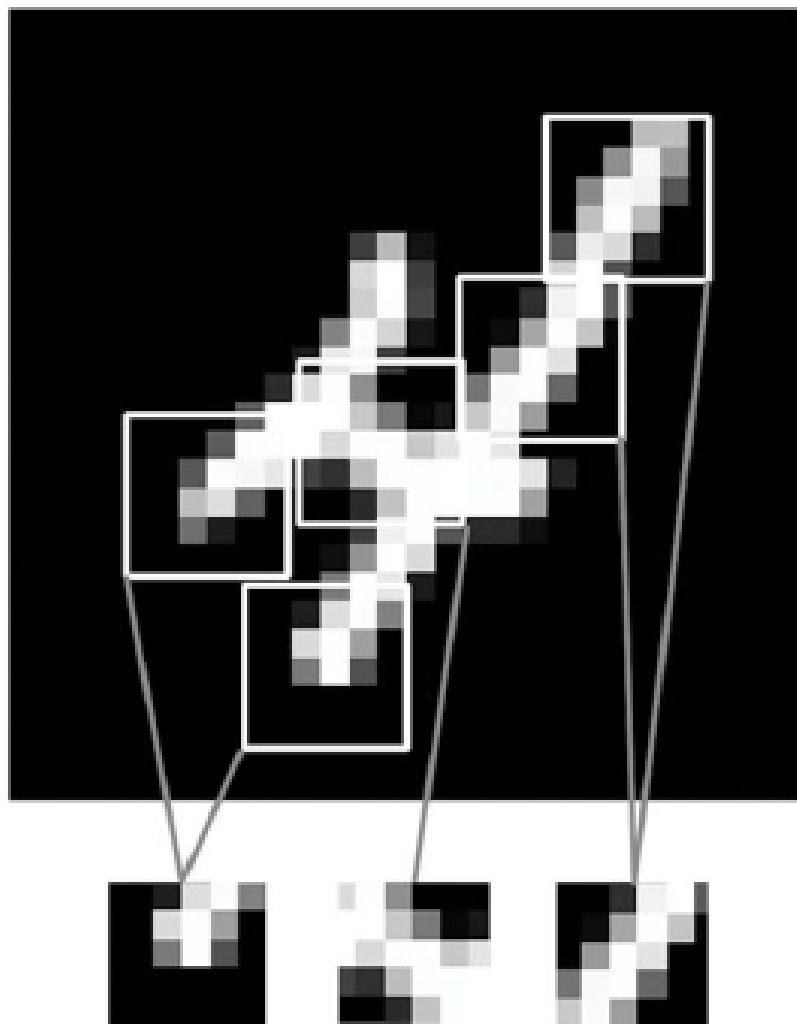


# Image tensors and feature maps

## Tensor ảnh và bản đồ đặc trưng

---

- Output của **Conv2D** và **MaxPooling2D** vẫn là tensor 3D: **height x width x channels**.
- Khi đi sâu hơn, **height/width thường giảm**, còn **channels thường tăng**.
- Mỗi channel có thể hiểu như một **feature map**: bản đồ cho biết một đặc trưng xuất hiện ở đâu.
- Số channel đầu ra là tham số **filters** trong `Conv2D(filters=...)`.



**Figure 8.1** Images can be broken into local patterns such as edges, textures, and so on.

Hình 8.1. Ảnh có thể được phân rã thành các mẫu cục bộ như cạnh và kết cấu.

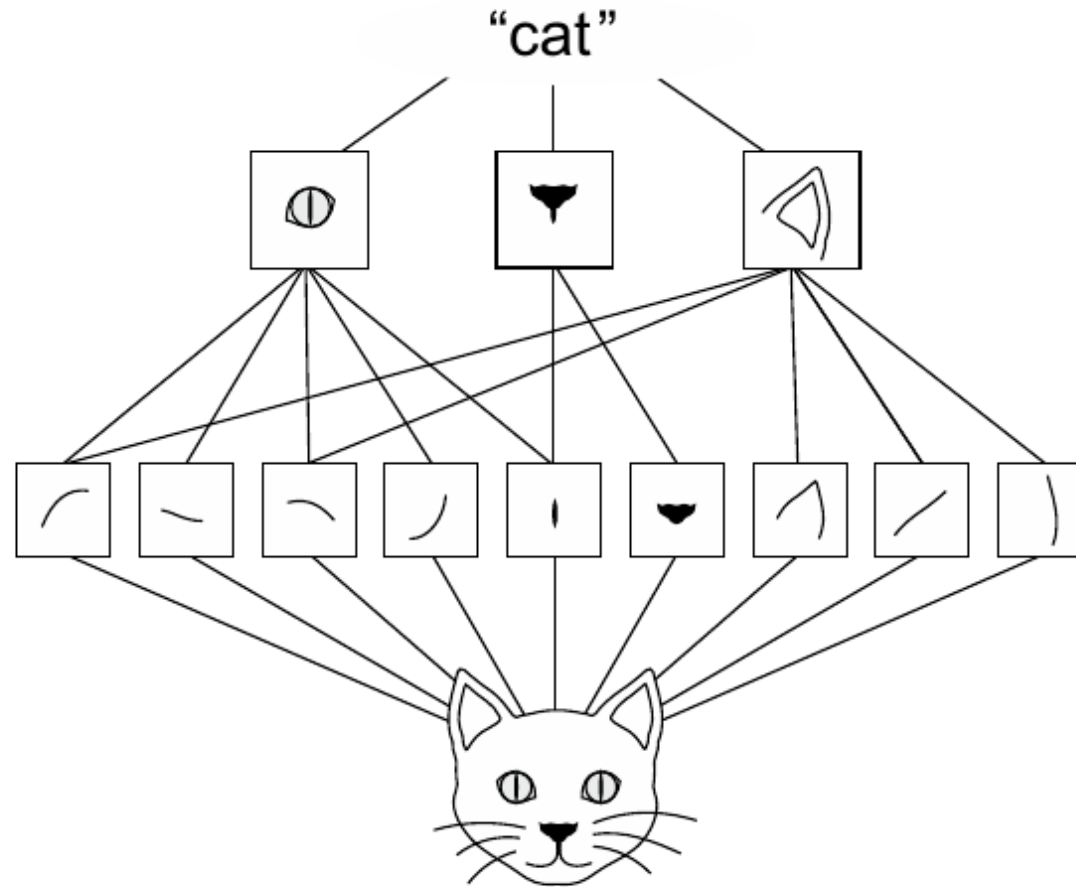
# Why convolution works well

## Vì sao convolution hiệu quả?

---

- Convnet học **local patterns** thay vì học quan hệ toàn cục giữa mọi pixel.
- Các pattern học được có tính **translation-invariant** (bất biến theo phép tịnh tiến): không phụ thuộc vào vị trí xuất hiện của mẫu trong ảnh.
- Ví dụ: nếu CNN học được pattern “tai mèo” ở góc trên bên trái, nó vẫn có thể nhận ra pattern đó khi “tai mèo” xuất hiện ở góc khác của ảnh.
- Convnet học **spatial hierarchies** (phân cấp đặc trưng theo không gian): cạnh/texture (kết cấu) -> bộ phận -> đối tượng.
- Ví dụ: CNN học từ **cạnh, nét cong, texture** ở tầng thấp, rồi kết hợp thành **mắt, tai, mũi**, cuối cùng thành khái niệm lớn như **mèo** hoặc **chó**.
- Nhờ hai đặc tính này, convnet thường **data-efficient** hơn Dense model trên bài toán ảnh.





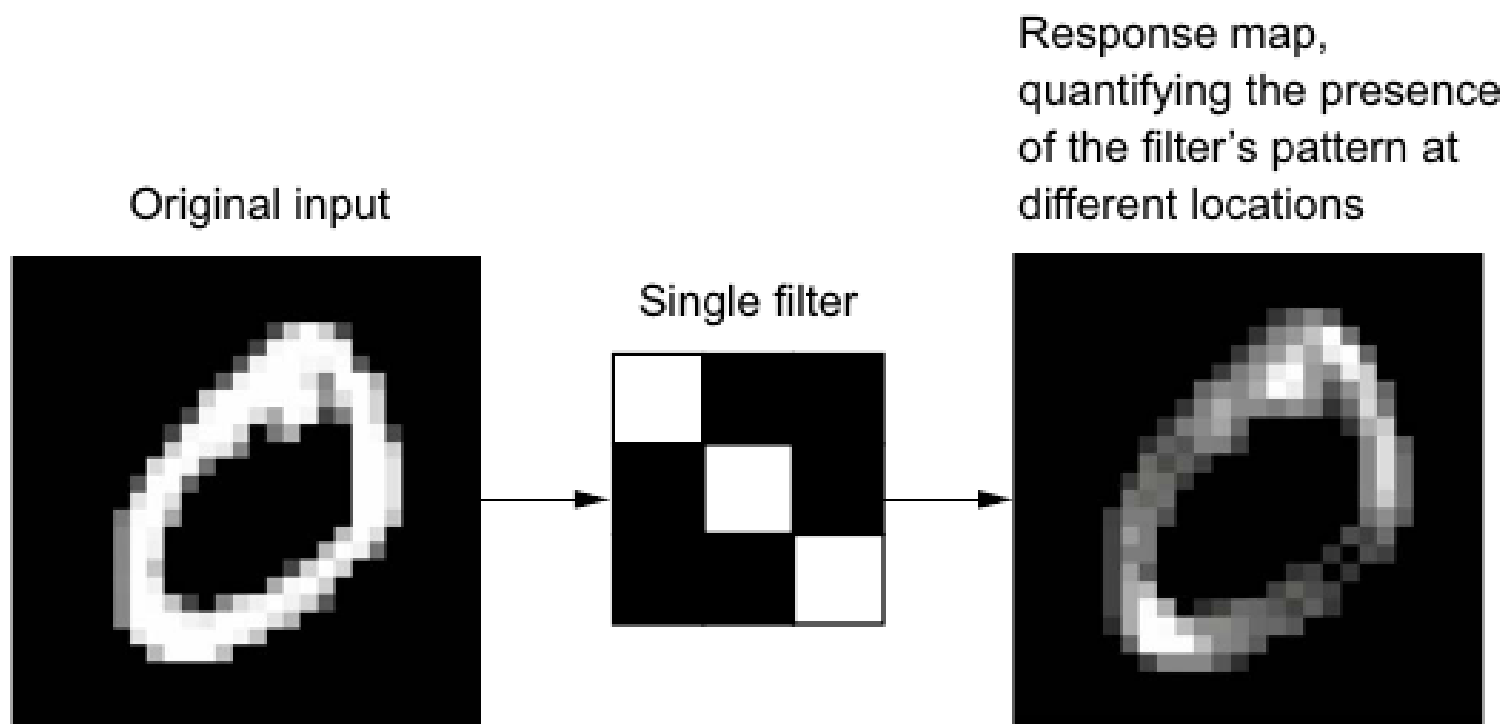
**Figure 8.2** The visual world forms a spatial hierarchy of visual modules: elementary lines or textures combine into simple objects such as eyes or ears, which combine into high-level concepts such as “cat.”

Hình 8.2. Thế giới thị giác có cấu trúc phân cấp theo không gian.

## Filters and response maps Bộ lọc và bản đồ đáp ứng

---

- Một **filter/kernel** là ma trận trọng số được trượt qua ảnh hoặc feature map.
- Mỗi filter tạo ra một **response map**: vị trí nào khớp pattern thì activation cao.
- Nếu `filters=32`, lớp convolution tạo ra **32 response maps** khác nhau.
- Các channel đầu ra không còn là RGB, mà là **mức hiện diện của các pattern**.



**Figure 8.3** The concept of a response map: a 2D map of the presence of a pattern at different locations in an input

Hình 8.3. Response map cho biết pattern xuất hiện ở đâu trong input.

## The convolution operation Phép tích chập hoạt động thế nào?

---

- Convolution trượt một cửa sổ nhỏ, thường **3 x 3** hoặc **5 x 5**, trên input feature map.
- Mỗi patch 3D được nhân với **kernel học được** để tạo vector đầu ra tại vị trí tương ứng.
- Cùng một kernel được dùng lại trên mọi vị trí: **weight sharing**.
- Kết quả là một output feature map có dạng **height x width x output\_depth**.

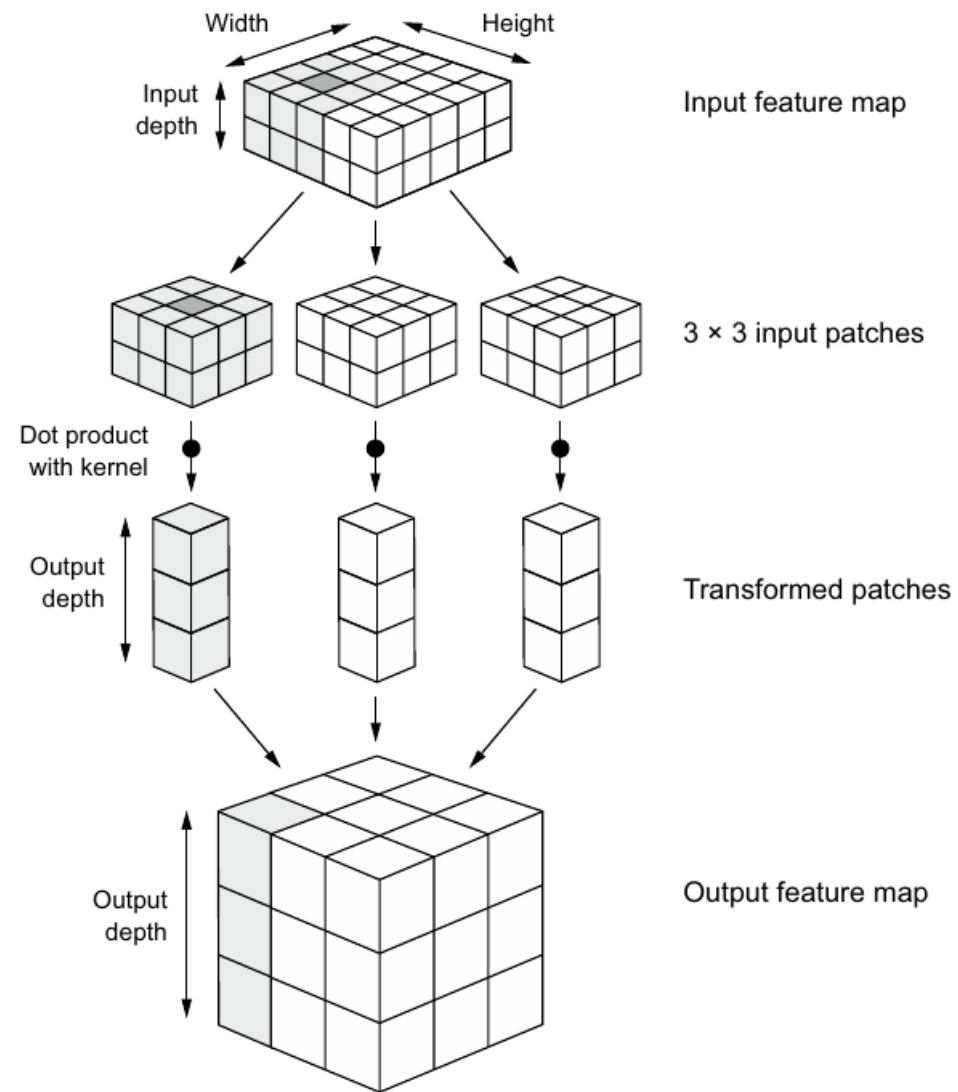


Figure 8.4 How convolution works

Hình 8.4. Cách convolution biến các patch (vùng con) đầu vào thành output feature map.

## Padding, strides, and output size

### Padding, stride và kích thước đầu ra

---

- **Border effects** làm output nhỏ hơn input khi không padding.
- **Padding** (đệm) thêm hàng/cột quanh input để giữ hoặc kiểm soát kích thước output.
- **Stride** (bước trượt) là khoảng cách giữa hai vị trí cửa sổ liên tiếp; stride lớn hơn làm output nhỏ hơn.
- Trong Keras, `padding="valid"` thường làm co kích thước; `padding="same"` giữ kích thước gần như input.

## Max pooling and downsampling

### Max pooling và giảm kích thước

---

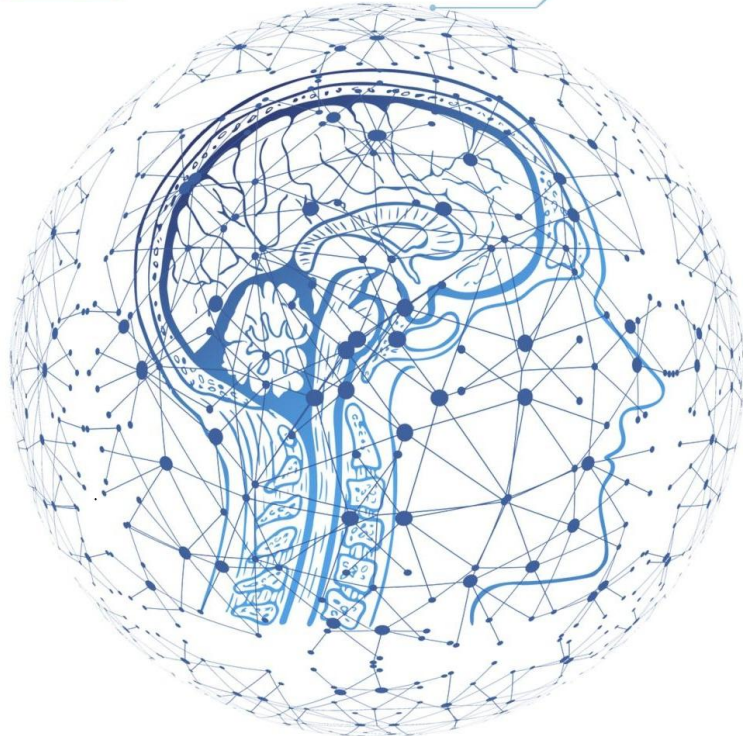
- **MaxPooling2D** lấy giá trị lớn nhất trong mỗi patch, thường **2 x 2** với stride 2.
- Mục tiêu: giảm số hệ số của feature map và giảm chi phí tính toán.
- Pooling giúp tầng sau nhìn được **vùng lớn hơn của ảnh gốc**.
- Max pooling thường hiệu quả vì giữ lại **mức hiện diện mạnh nhất** của pattern.



# DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Training a convnet from scratch  
on a small dataset  
Huấn luyện convnet từ đầu  
trên tập dữ liệu nhỏ



## The small-data setting Bối cảnh dữ liệu ít

---

- Trong thực tế, bài toán ảnh thường chỉ có **vài trăm đến vài nghìn ảnh**.
- Ví dụ: phân loại **Dogs vs. Cats** với 5,000 ảnh được chia nhỏ.
- Chiến lược đầu tiên: huấn luyện một **small convnet from scratch** để lấy baseline.
- Vấn đề chính khi dữ liệu ít: **overfitting**.

## Dogs vs. Cats dataset Bộ dữ liệu chó và mèo

---

- Dataset gốc có **25,000 ảnh**: 12,500 mèo và 12,500 chó.
- Tạo tập nhỏ: **2,000 train, 1,000 validation, 2,000 test**.
- Mỗi split cân bằng số ảnh mèo/chó, nên **accuracy** là metric phù hợp.
- Ảnh là JPEG màu, đa dạng về **kích thước, nền, ánh sáng, tư thế**.



**Figure 8.8** Samples from the Dogs vs. Cats dataset. Sizes weren't modified: the samples come in different sizes, colors, backgrounds, etc.

Hình 8.8. Mẫu ảnh Dogs vs. Cats có kích thước, màu sắc và bối cảnh khác nhau.

## Building a baseline convnet Xây dựng convnet baseline

---

- Model dùng chuỗi **Conv2D + MaxPooling2D** giống ví dụ MNIST nhưng lớn hơn.
- Input được resize về **180 x 180 x 3**.
- Đầu ra là một neuron **sigmoid** vì đây là bài toán binary classification.
- Loss phù hợp: **binary\_crossentropy**; metric: **accuracy**.

# Data preprocessing pipeline

## Pipeline tiền xử lý dữ liệu

---

- Bước xử lý ảnh: đọc file → decode JPEG → tensor float → resize → batch.
- Keras hỗ trợ bằng ``image_dataset_from_directory()``.
- Hàm này tự suy ra class từ **tên thư mục con** như ``cat/`` và ``dog/``.
- ``tf.data.Dataset`` có thể đưa trực tiếp vào ``model.fit()``.



# Baseline training and overfitting

## Huấn luyện baseline và overfitting

---

- Baseline đạt validation accuracy khoảng **70-75%**, nhưng nhanh chóng overfit.
- **Training accuracy** tiếp tục tăng gần 100%.
- **Validation accuracy** chững lại, còn **validation loss** tăng.
- Dấu hiệu này cho thấy model học thuộc train set thay vì học quy luật tổng quát.

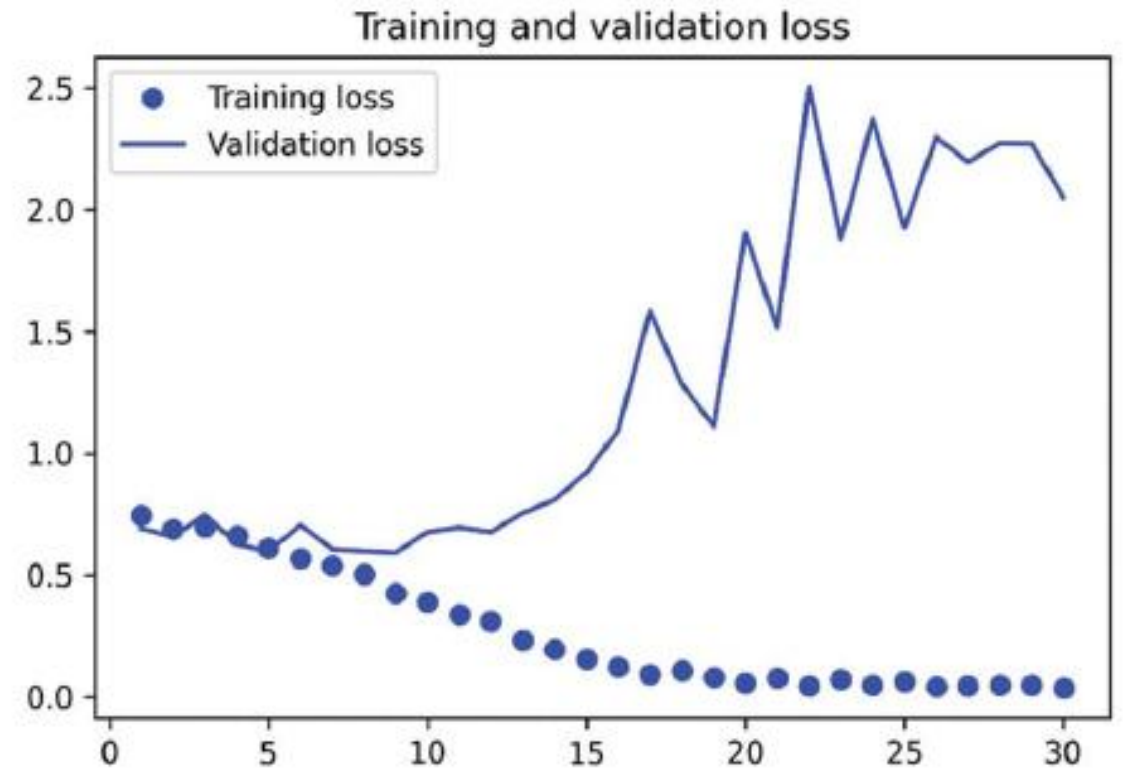
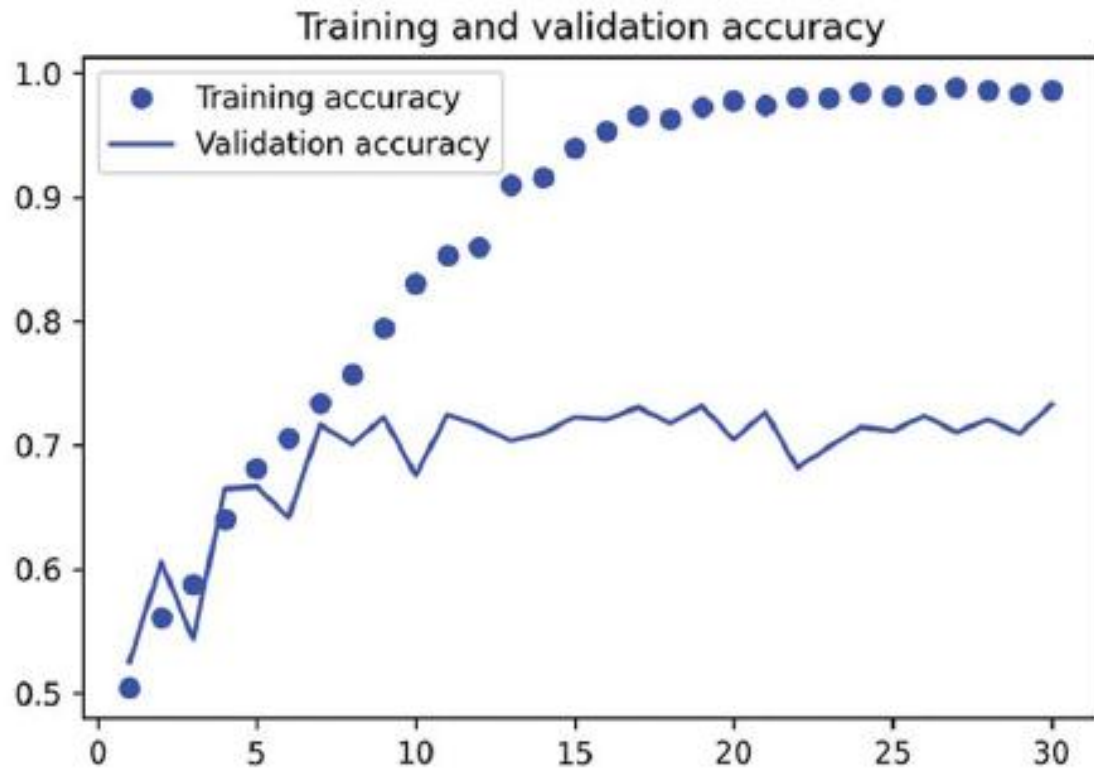


Figure 8.9 Training and validation metrics for a simple convnet

Hình 8.9. Metric train/validation của convnet đơn giản cho thấy overfitting.



## Data augmentation Tăng cường dữ liệu

---

- **Data augmentation** là kỹ thuật tạo ra các phiên bản biến đổi nhẹ của dữ liệu huấn luyện, nhưng vẫn giữ nguyên nhãn.
- Ví dụ: **flip, rotation, zoom, translation**, thay đổi nhỏ về hình học.
- Mục tiêu: model không bao giờ thấy đúng một ảnh y hệt, nên khó học thuộc hơn.
- Trong Keras, augmentation có thể đặt thành các layer đầu của model.

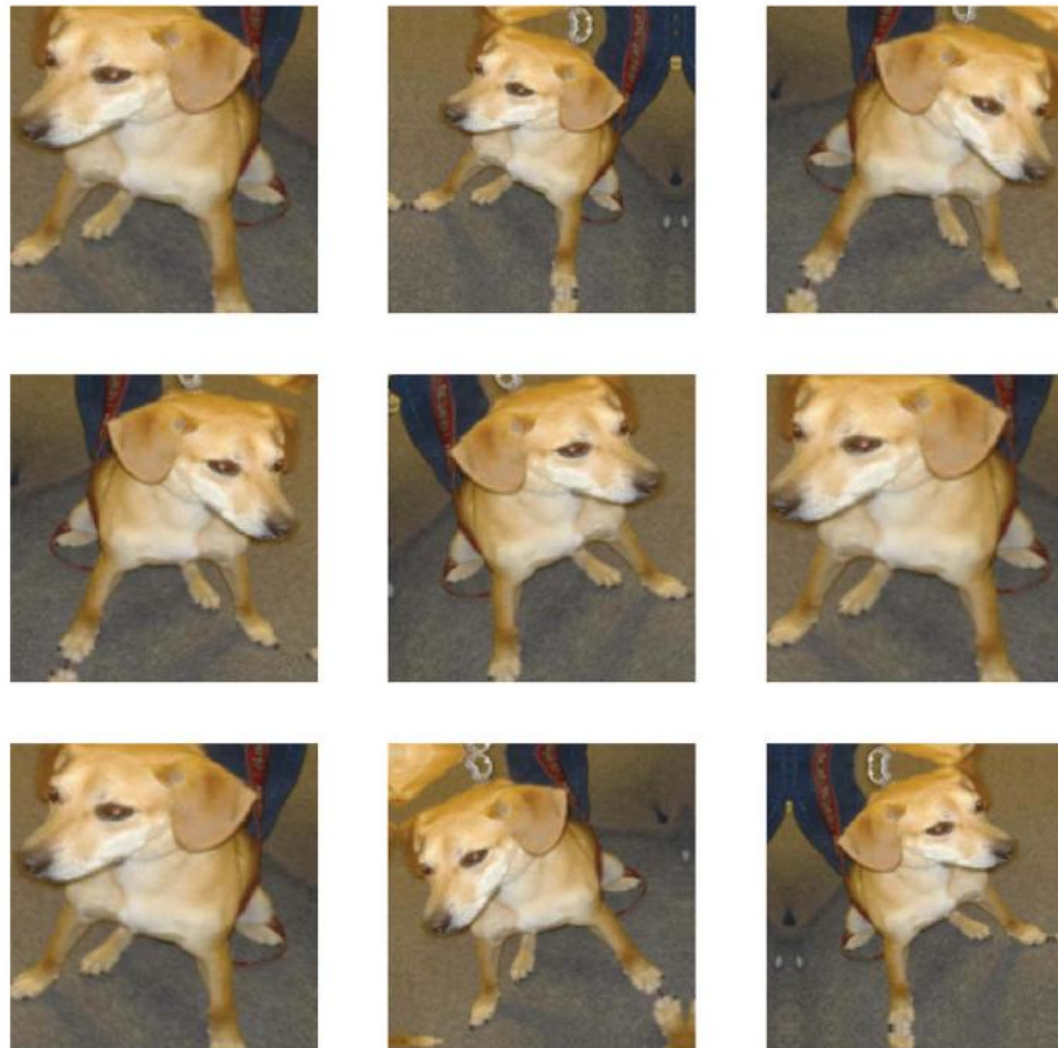


Figure 8.10 Generating variations of a very good boy via random data augmentation

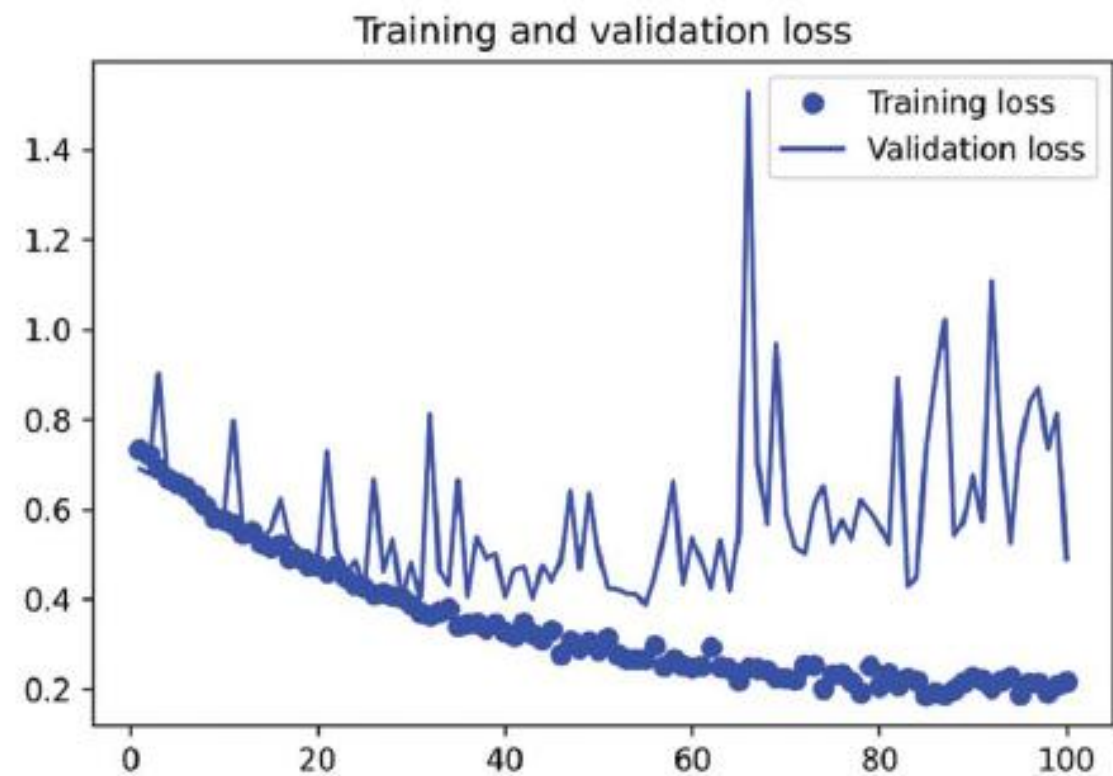
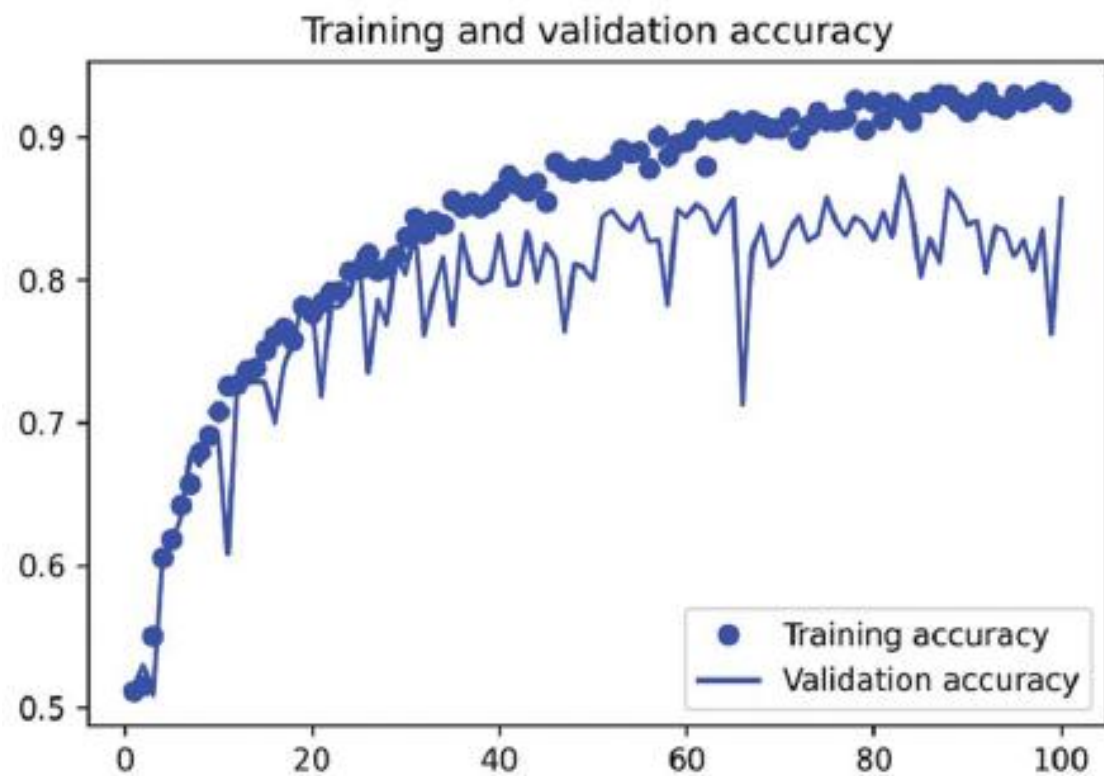
Hình 8.10. Các biến thể ngẫu nhiên của cùng một ảnh nhờ data augmentation.

# Regularization with augmentation and dropout

## Regularization bằng augmentation và dropout

---

- **Data augmentation** làm tăng sự đa dạng quan sát được trong train set.
- **Dropout** giúp giảm phụ thuộc vào một số activation cụ thể.
- Data augmentation xử lý overfitting ở mức dữ liệu; dropout xử lý ở mức biểu diễn bên trong model.
- Kết hợp hai kỹ thuật giúp overfitting xảy ra muộn hơn.
- Trong sách, accuracy test tăng lên khoảng **83.5%** với model từ đầu.



**Figure 8.11** Training and validation metrics with data augmentation

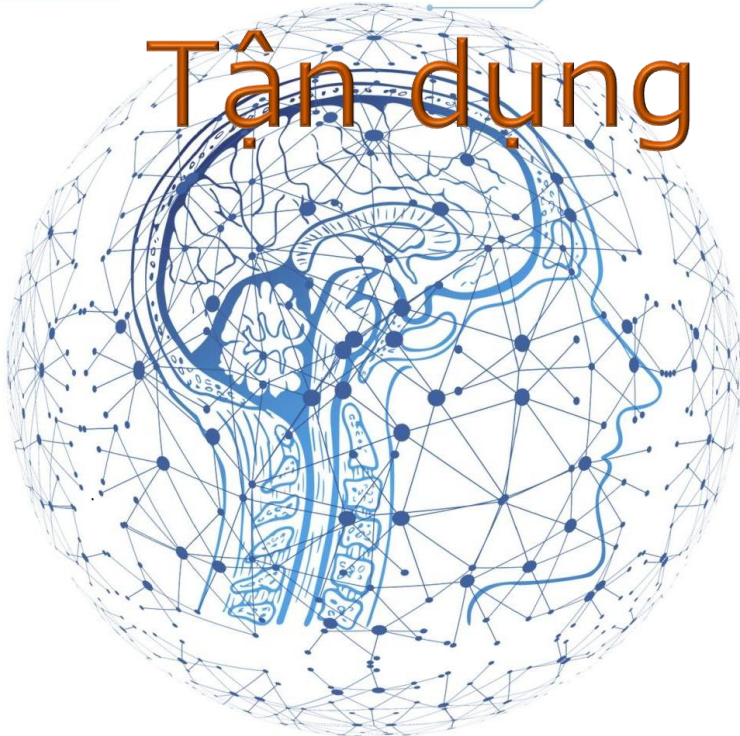
Hình 8.11. Data augmentation giúp validation metric cải thiện và overfitting muộn hơn.

# DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

Leveraging a pretrained model

Tận dụng mô hình đã huấn luyện sẵn



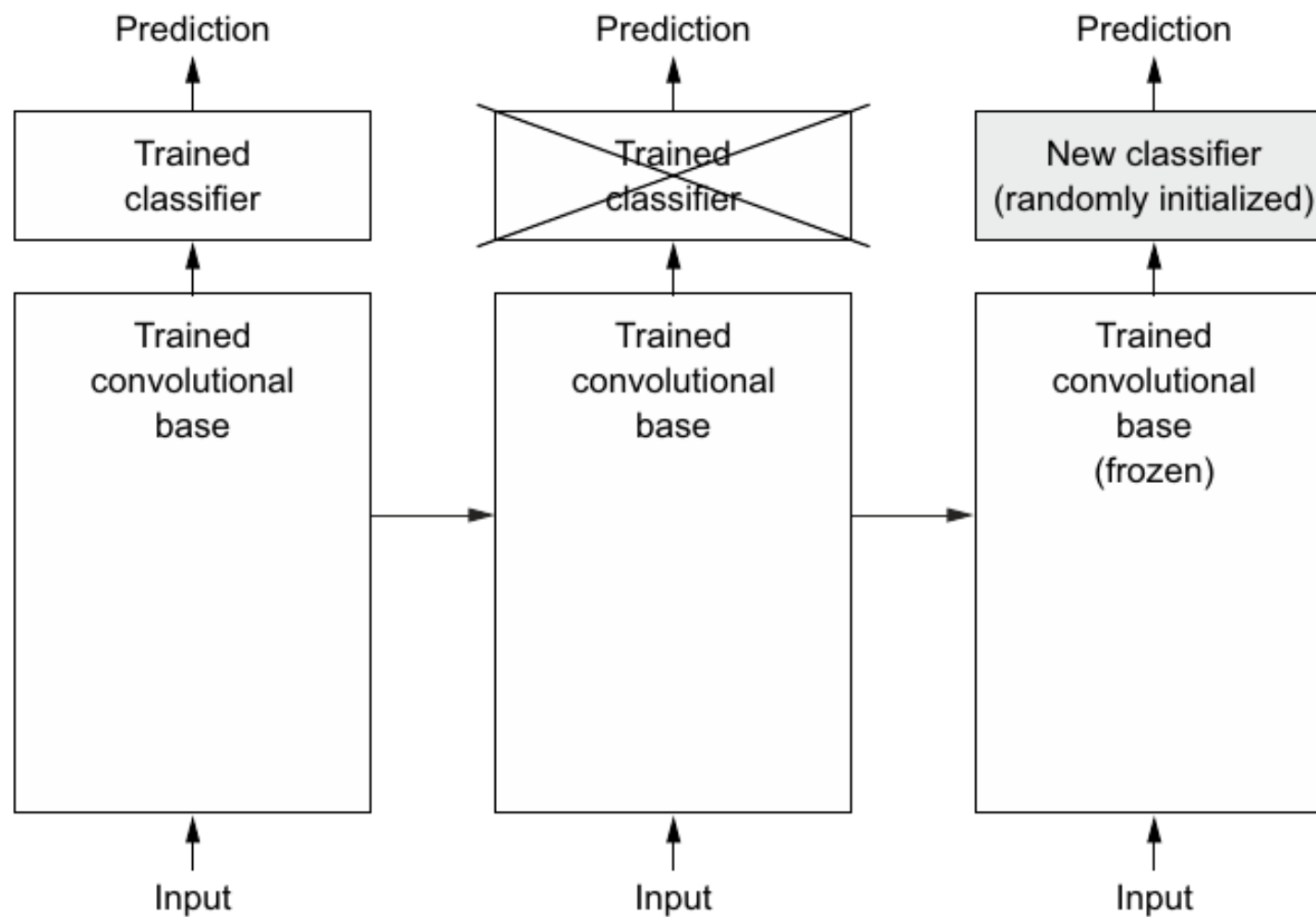


# Pretrained models and feature reuse

## Mô hình pretrained và tái sử dụng đặc trưng

---

- **Pretrained model** đã được huấn luyện trên một dataset lớn, ví dụ ImageNet.
- Nếu dataset gốc đủ lớn và tổng quát, feature học được có thể dùng cho bài toán mới.
- ImageNet có khoảng **1.4 triệu ảnh** và **1,000 lớp**.
- Sách dùng **VGG16** vì kiến trúc dễ hiểu và gần với convnet đã học.



**Figure 8.12** Swapping classifiers while keeping the same convolutional base

Hình 8.12. Giữ convolutional base đã học và thay classifier mới.



## VGG16 convolutional base

### Convolutional base của VGG16

---

- Dùng ``keras.applications.vgg16.VGG16`` với ``weights="imagenet"``.
- ``include_top=False`` bỏ classifier 1,000 lớp của ImageNet.
- ``input_shape=(180, 180, 3)`` phù hợp ảnh Dogs vs. Cats đã resize.
- Convolutional base cuối cùng tạo feature map khoảng **5 x 5 x 512**.

# Two feature extraction workflows

## Hai quy trình trích xuất đặc trưng

---

### ■ **Fast workflow:**

Ảnh train

→ conv\_base pretrained

→ lưu features ra mảng NumPy

→ train classifier nhỏ trên features đó

■ Chạy **conv\_base** một lần để lưu features, rồi train **classifier** nhỏ.

■ Ưu điểm: rất nhanh, kể cả trên CPU; nhược điểm: khó dùng data augmentation động.

### ■ **Integrated workflow:**

Ảnh train

→ data augmentation

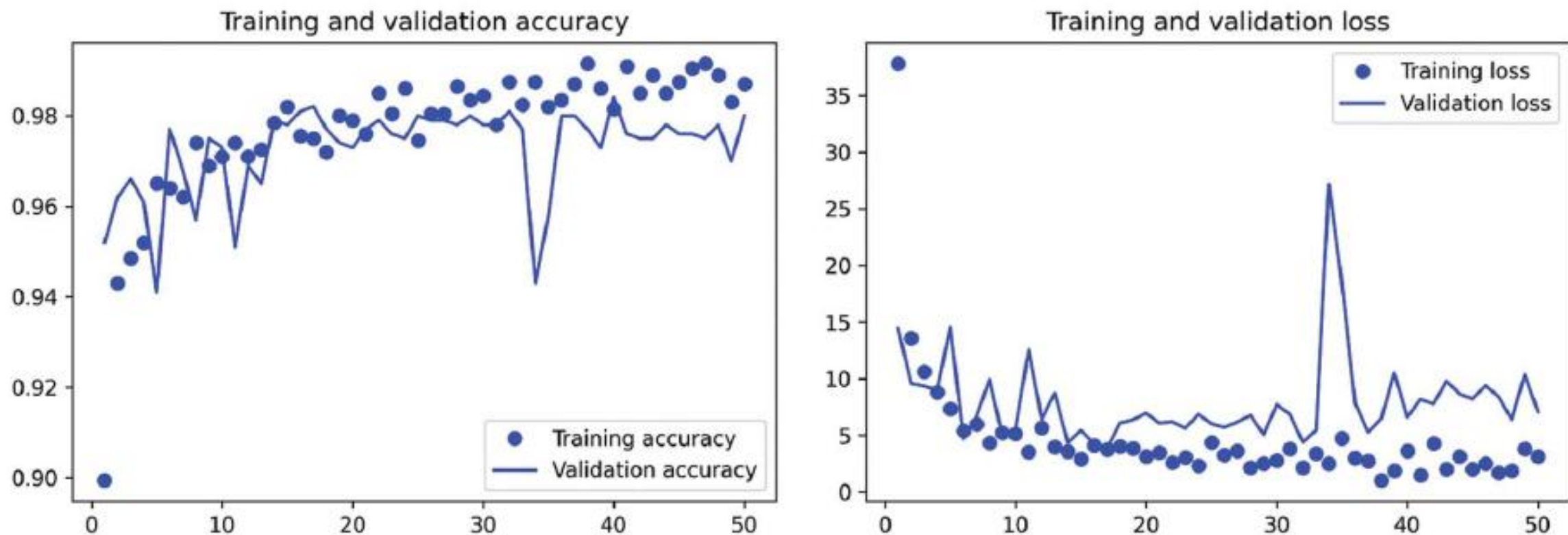
→ conv\_base frozen

→ classifier mới

→ output

■ Đặt **conv\_base frozen** trong model, thêm **augmentation** và **classifier**.

■ Ưu điểm: tận dụng augmentation; nhược điểm: tốn tính toán hơn vì mỗi epoch chạy qua conv\_base.



**Figure 8.14** Training and validation metrics for feature extraction with data augmentation

Hình 8.14. Feature extraction với data augmentation đạt validation accuracy trên 98%.

## Fine-tuning a pretrained model

### Tinh chỉnh mô hình pretrained

---

- **Fine-tuning** mở khóa một vài tầng cuối cùng của conv\_base đã frozen.
- Quy trình: thêm classifier → freeze base → train classifier → unfreeze top layers → train chung.
- Chỉ fine-tune tầng trên vì chúng học **khái niệm trừu tượng và đặc thù hơn**.
- Dùng learning rate nhỏ để tránh phá hỏng các biểu diễn đã học.

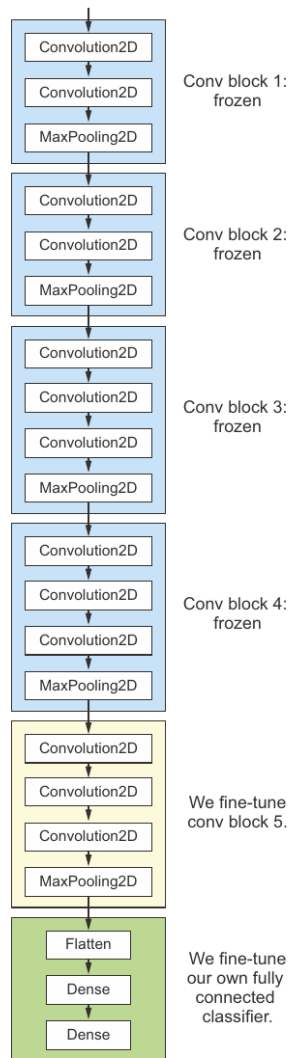


Figure 8.15 Fine-tuning the last convolutional block of the VGG16 network

Ảnh đầu vào

→ conv\_base pretrained

- block1 frozen

- block2 frozen

- block3 frozen

- block4 frozen

- **block5 trainable**

→ **classifier mới**

→ output mèo/chó

Hình 8.15. Fine-tuning block convolution cuối của VGG16 và classifier mới.

## Chapter Summary

### Tổng kết chương

---

- **Convnets** là mô hình rất phù hợp cho computer vision vì học local patterns và spatial hierarchies.
- Với dữ liệu ít, có thể train convnet từ đầu nhưng vấn đề chính là **overfitting**.
- **Data augmentation** là kỹ thuật mạnh để regularize mô hình ảnh.
- **Feature extraction** tái sử dụng convolutional base pretrained để đạt kết quả cao trên dataset nhỏ.
- **Fine-tuning** điều chỉnh thêm các tầng trên cùng để cải thiện hiệu năng.

